



Documentación Técnica



Integrantes:

- Lucas Frey
- Gaston Paez
- Nicolas Perez

Patrocinador:

- Valeria Espindola

Fecha de inicio: 1 de noviembre de 2025

Fecha de finalización: Abril 2026



Documentación Técnica	3
Tecnologías utilizadas	4
Front-End	4
Backend	4
Base de datos	4
Seguridad y Autenticación	5
Diagrama de Componentes	10
Diagrama de Entidad Relación	11
Diagrama de Clases	12
Diagrama de Secuencia	19
Caso 1: Creación de campaña solidaria	19
Caso 2: Realización de una donación	21



Documentación Técnica

SolidApp es una aplicación web cliente-servidor enfocada en promover la solidaridad y la donación de bienes para ayudar a organizaciones y entidades necesitadas. La aplicación permite a los usuarios registrarse y colaborar donando a campañas activas.

Las empresas, por su parte, pueden incentivar a los usuarios a donar mediante la publicación de cupones o beneficios canjeables por puntos, promoviendo así la participación activa en las donaciones.

El sitio web fue diseñado utilizando una arquitectura por capas, lo que implica una clara separación de responsabilidades entre frontend y backend. Si bien ambos forman parte del mismo repositorio, mantienen una división lógica que favorece la mantenibilidad, escalabilidad y estabilidad del sistema. La persistencia se gestiona mediante una base de datos relacional.

En el backend se adoptó la arquitectura modular propia de NestJS, basada en módulos independientes. Por cada entidad del dominio, se organiza el código con la siguiente estructura:

- `.entity.ts`: definición de entidades de TypeOrm.
- `.controller.ts`: controlador HTTP.
- `.service.ts`: lógica de negocio.
- `.module.ts`: módulos de configuración e integración de NestJS.
- `.dto.ts`: Validadores de datos.
- `.enum.ts`: Conjunto de Constantes.



Tecnologías utilizadas

Front-End

- Framework: NextJS.
- Lenguaje: TypeScript.
- Biblioteca UI: React, SweetAlert2.
- Estilos: CSS + TailWind.

Backend

- Framework: NestJS.
- Lenguaje: TypeScript.
- ORM: TypeOrm.
- Autenticación: JSON Web Token.
- Testing: Jest y SonarQube.
- Multer
- Bcrypt
- passport-jwt

Base de datos

- Motor: Microsoft Sql Server (Express) y Microsoft SQL Management Studio.
- Tipo: Relacional.
- Persistencia

Seguridad y Autenticación

En cuanto a la seguridad de nuestra aplicación, utilizamos JSON Web Token y permisos basados en roles a través de Guards y Decorators implementados junto al módulo de autenticación (auth). Estos son utilizados en los controllers de la aplicación para proteger los endpoints de request generados por entidades que no sean las esperadas por esos endpoints.

La autenticación se lleva a cabo por el módulo auth. Al momento de que el usuario haga login, se utilizan las credenciales (email y contraseña) de este para realizar las validaciones:

- Las contraseñas se almacenan y se comparan a través del uso de hasheo por bcrypt.
- Se comparan el email ingresado con los almacenados, para determinar la existencia del usuario.

Si el login es exitoso, se genera un token (JWT) con los siguientes datos:

- . id del usuario.
- . rol.
- . gestion (Campo que indica si pertenece a una empresa u organización)
- . gestionId (Id de la organización o empresa)

Este se envía en cada solicitud mediante el header Authorization: Bearer {token}.

Los roles implementados en la aplicación están divididos en roles principales y secundarios:

Principales:

- USUARIO: Puede donar
- ADMIN: Administra Usuarios, Empresas, Organizaciones, Beneficios y Campañas Solidarias. Genera invitaciones para Empresas y Organizaciones.
- COLABORADOR: Usuarios que administran Empresas u Organizaciones.

Secundarios:

- GESTOR: Admin de la Empresa u Organización.
- MIEMBRO: Empleado de la Empresa u Organización.
- ENTIDAD: Organización o Empresa (Utilizado para invitaciones)

Son 2 decorators distintos los que utilizan estos roles:

- @Auth(<Rol Principal>): Determina el acceso según el rol principal.
- @AuthRelacion(<Rol Secundario>): Determina el acceso según el rol secundario.

Modelo de Datos

Nuestro modelo de Datos es relacional y diseñado utilizando SQL SERVER EXPRESS, a través de SQL Server Management Studio. Mientras que la persistencia de datos de nuestra DB, utiliza TypeORM.

Las entidades del sistema son:

- **Usuarios:**

- . Representa a una persona registrada en el sitio.
- . Puede ser Admin, Usuario o Colaborador.
- . Un Usuario puede realizar donaciones a diferentes campañas asociadas a diferentes Organizaciones.
- . Un Usuario puede canjear y usar diferentes cupones asociados a diferentes empresas.
- . Un Usuario al donar, ya participa de un ranking.
- . El Admin del sitio, puede habilitar/deshabilitar, usuarios, empresas y organizaciones. Y aceptar/rechazar beneficios y campañas solidarias
- . El Colaborador, puede llevar a cabo varias funciones. Del lado de la Empresa, según su rol secundario, puede crear y actualizar cupones/beneficios, y también puede invitar a más colaboradores.
- . Del lado de la Organización, según su rol, puede crear y actualizar Campañas Solidarias, puede aceptar/rechazar donaciones realizadas por usuarios y también puede invitar a más colaboradores.

Relaciones:

- . Usuario 1 : M Donaciones
- . Usuario 1 : 1 Ranking
- . Usuario 1 : M Usuario Beneficio
- . Usuario 1 : 1 Contacto
- . Usuario 1 : 1 Dirección
- . Usuario 1 : M Organizacion Usuario
- . Usuario 1 : N Empresa Usuario

- **Contacto:**

- . Representa la lista de contacto de los usuarios (correo, teléfono).

- **Direcciones**

- . Representa la lista de dirección de los usuarios.

- **Empresa**

- . Representa a las entidades empresas que están registradas en el sitio.

Relaciones:

- . Empresa 1 : M Empresa Usuario
 - . Empresa 1 : M Beneficios
 - . Empresa 1 : 1 Contacto
 - . Empresa 1 : 1 Dirección

- **Organización**

- . Representa a las entidades organizaciones que están registradas en el sitio.

Relaciones:

- . Organizacion 1 : M Organizacion Usuario
 - . Organizacion 1 : M Campañas
 - . Organizacion 1 : 1 Contacto
 - . Organizacion 1 : 1 Dirección

- **Empresa Usuario**

- . Representa la lista de usuarios que pertenecen a una empresa

Relaciones:

- . Empresa Usuario N : 1 Usuario
 - . Empresa Usuario N : 1 Empresa

- **OrganizacionUsuario**

- . Representa a la lista de usuarios que pertenecen a una organización.

Relaciones:

- . Organización Usuario N : 1 Usuario
 - . Organización Usuario N : 1 Usuario

- **Beneficios**

- . Representa la lista de Beneficios que crean las empresas

Relaciones:

- . Beneficios N : 1 Empresa

- **Usuarios Beneficios**

- . Representa la lista de beneficios asociados (canjeados) por los usuarios

Relaciones:

- . Usuarios Beneficios N : 1 Beneficios
 - . Usuarios Beneficios N : 1 Usuario

- **Campañas**

- . Representa todas las campañas creadas por organizaciones

Relaciones:

- . Campañas N : 1 Organización
 - . Campañas 1 : M Donaciones
 - . Campañas 1 : M Imágenes campaña

- **Donaciones**

- . Representa todas las donaciones hechas por usuarios y dirigidas a las organizaciones correspondientes.

Relaciones:

- . Donaciones M : 1 Organización
 - . Donaciones M : 1 Usuario

- **ImágenesCampaña**

- . Representa todas las imágenes asociadas a una Campaña Solidaria

Relaciones:

- . ImágenesCampaña M : 1 Campaña

- **Invitaciones**

- . Representa todas las invitaciones realizadas por usuarios dentro del sitio

Relaciones:

- . Invitaciones M : 1 Empresa
 - . Invitaciones M : 1 Organización
 - . Invitaciones M : 1 Invitador
 - . Invitaciones 1 : 1 Usuario

- **Ranking**

- . Representa la lista del ranking (de puntos ganados) en el sitio

Relaciones:

- . Ranking 1 : 1 Usuario



Documentación de la API

La API desarrollada dispone de documentación interactiva implementada mediante Swagger, herramienta que facilita la exploración y prueba de los distintos endpoints. A través de esta interfaz es posible consultar los métodos disponibles, parámetros de entrada, esquemas de datos y respuestas esperadas para cada recurso.

Para acceder a dicha documentación, se debe ejecutar previamente el backend en un entorno local. Una vez en funcionamiento, se podrá acceder a la misma a través de la siguiente dirección:

<http://localhost:3001/api/>

Diagrama de Componentes

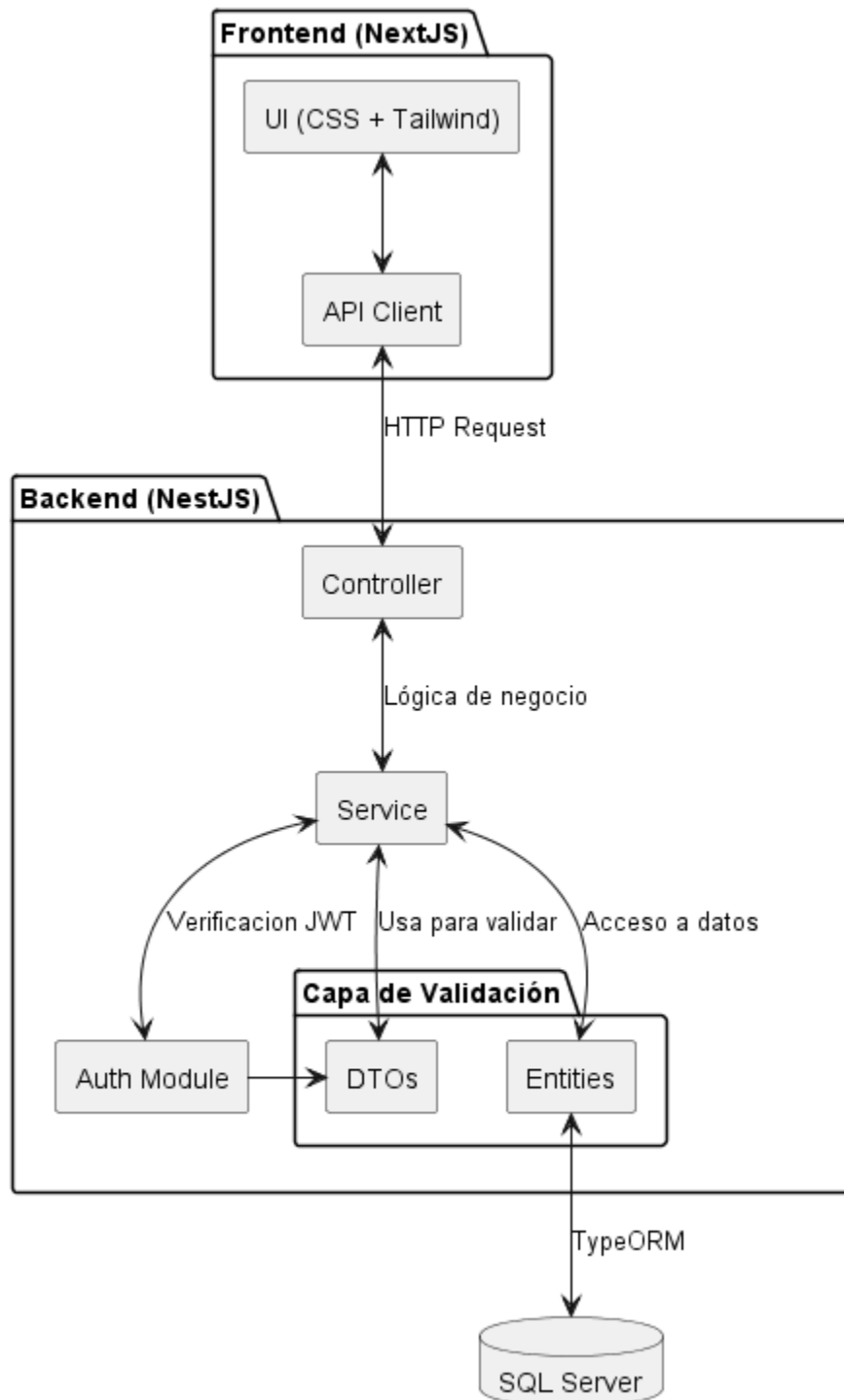
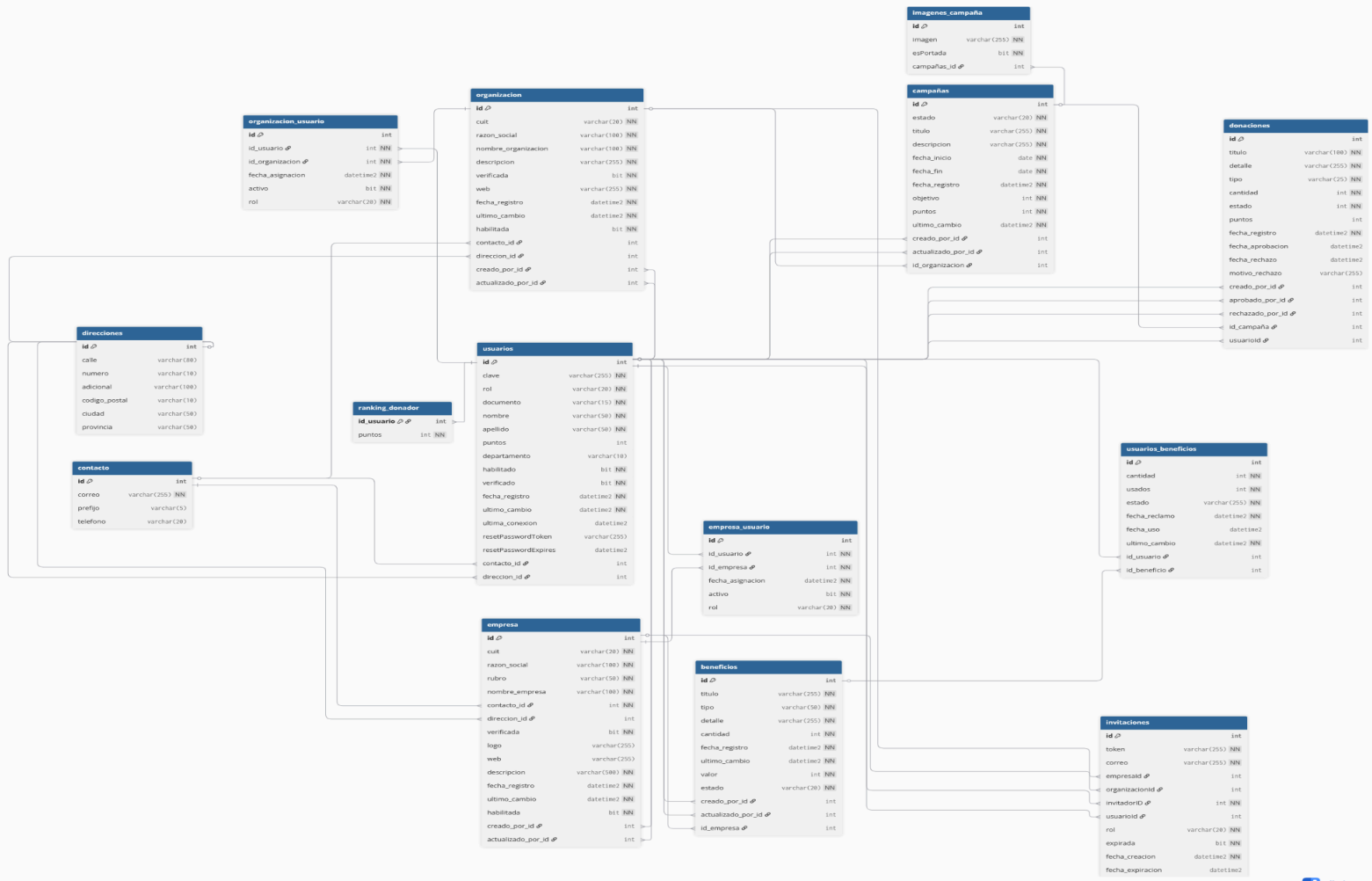


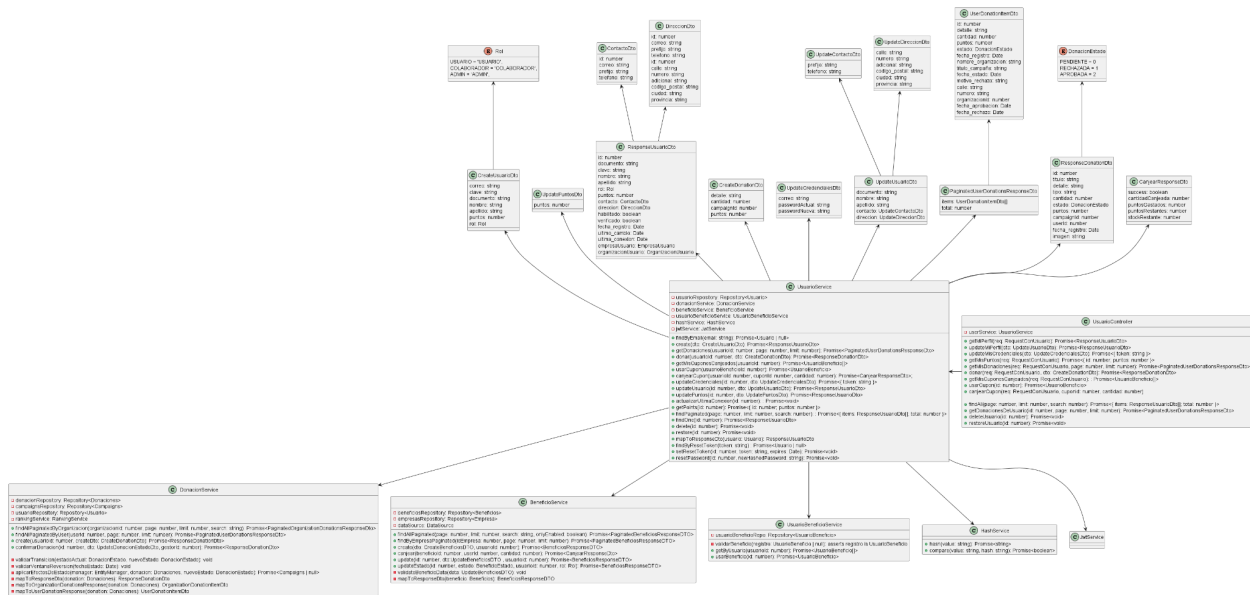
Diagrama de Entidad Relación



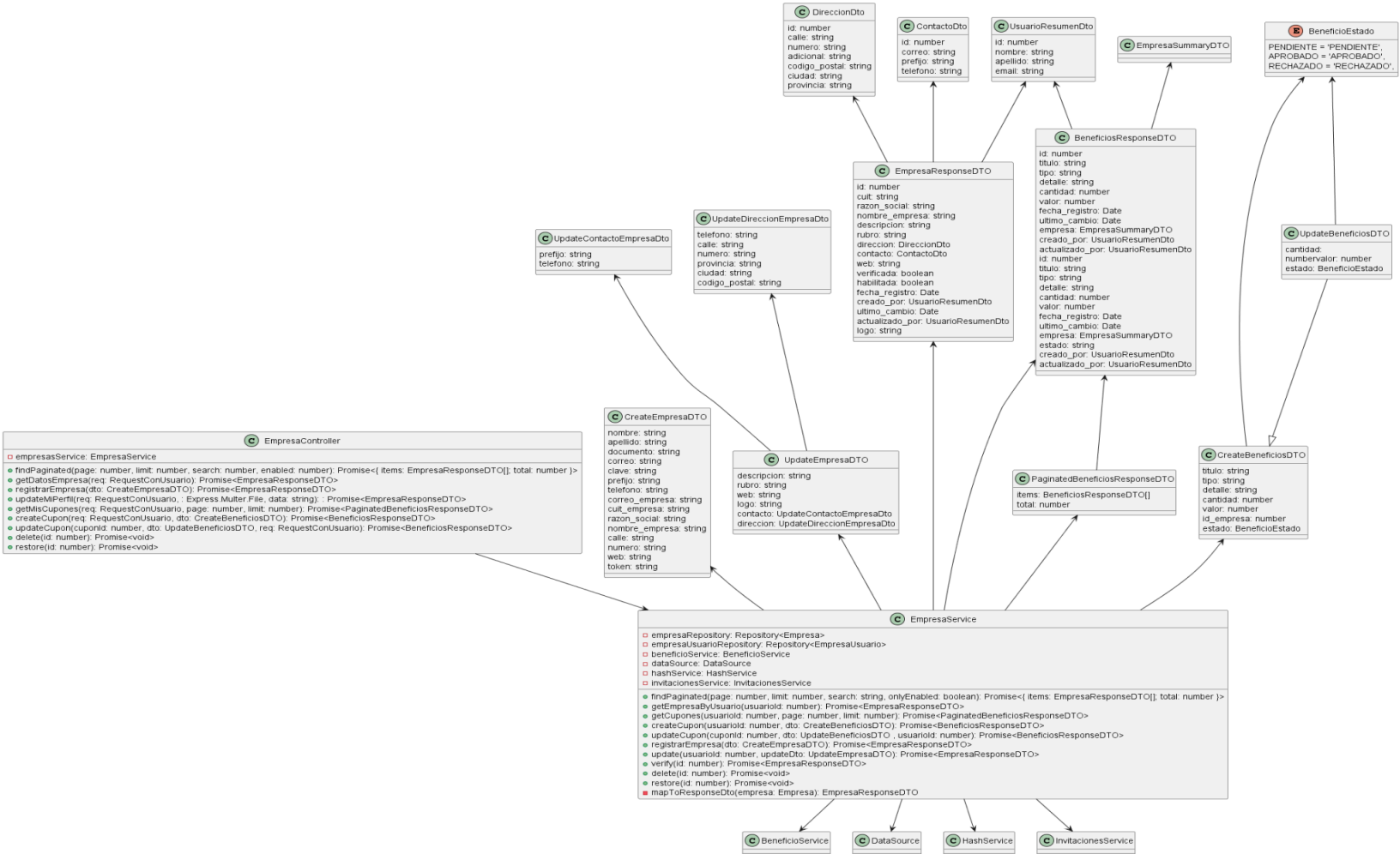
[Ver Diagrama de forma online.](#)

Diagrama de Clases

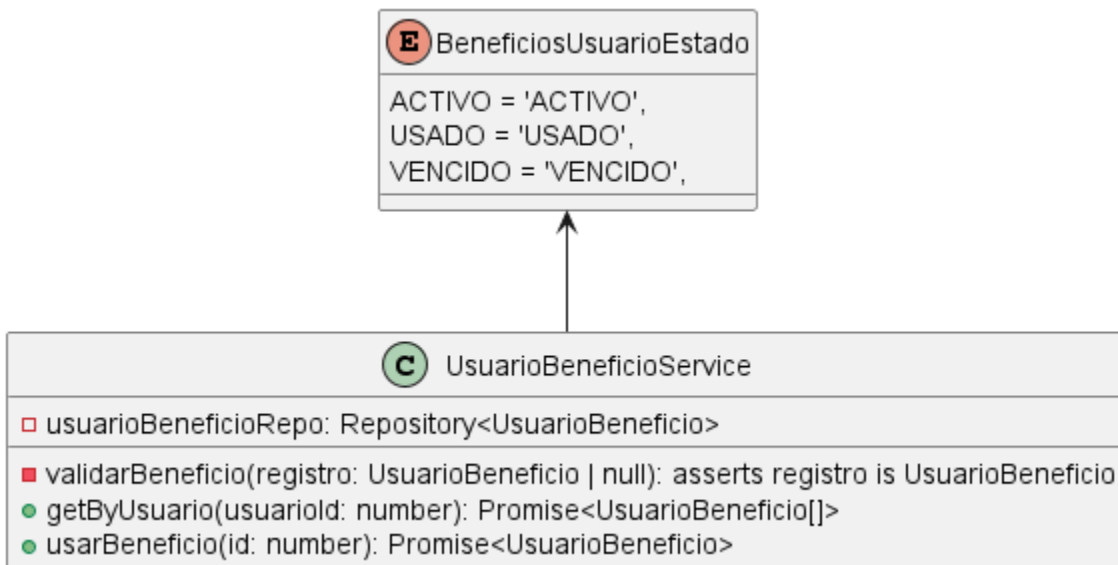
Usuario Service



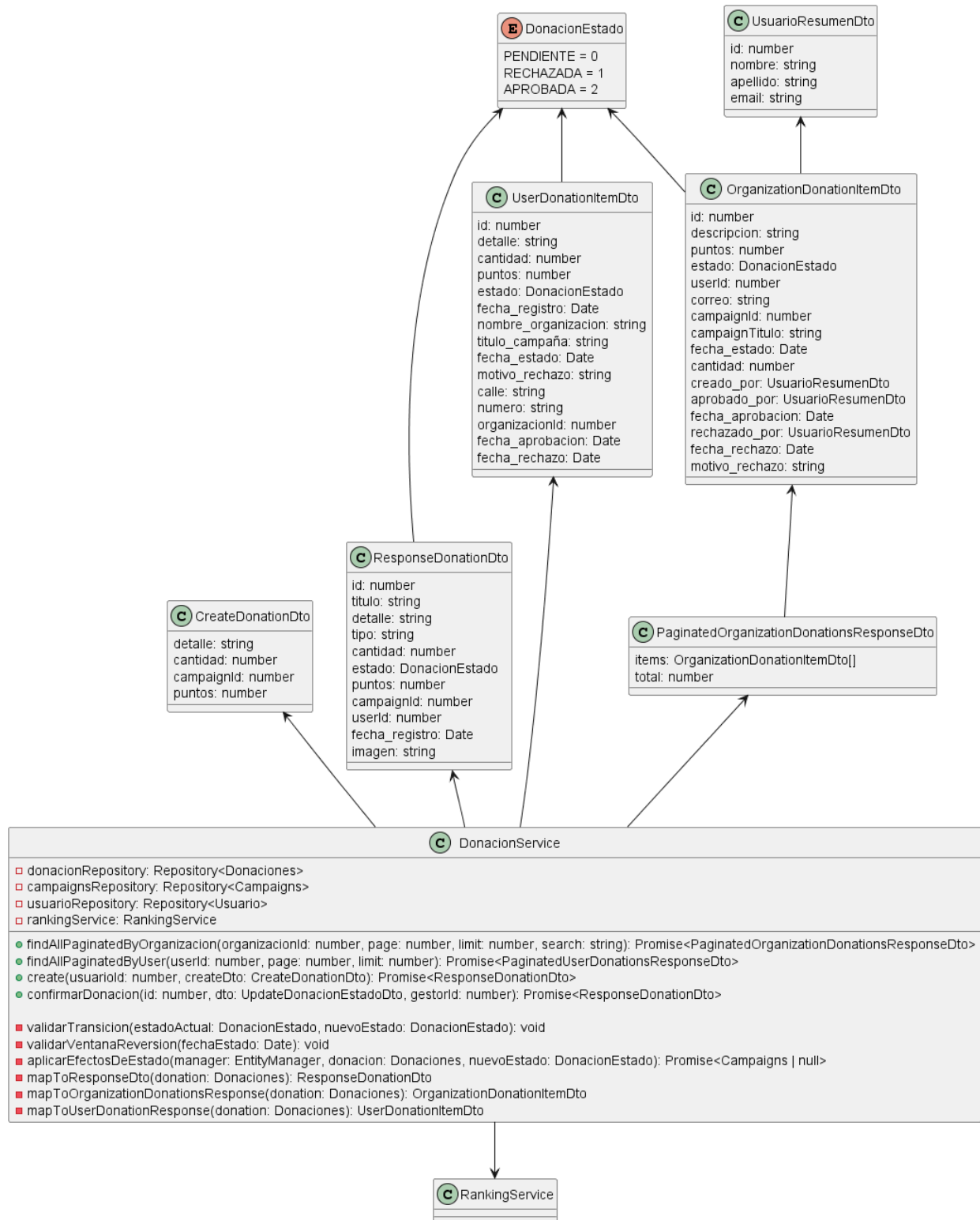
Empresa Service



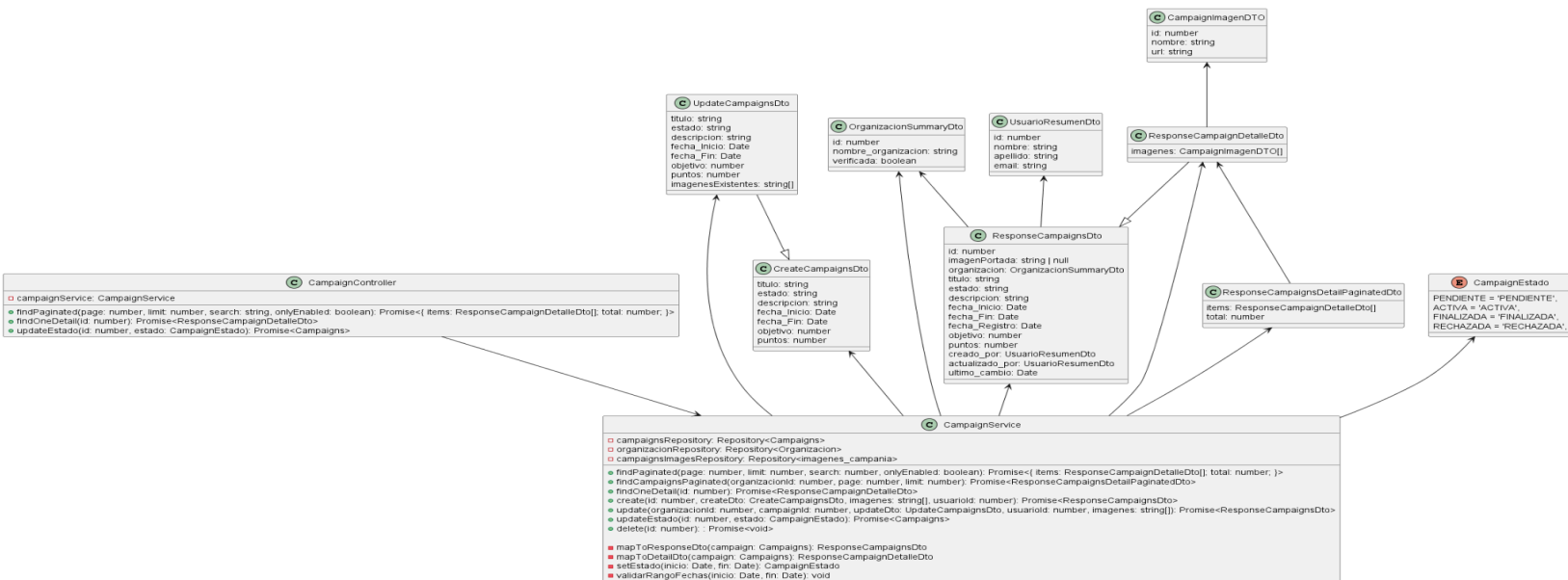
Usuario beneficio service



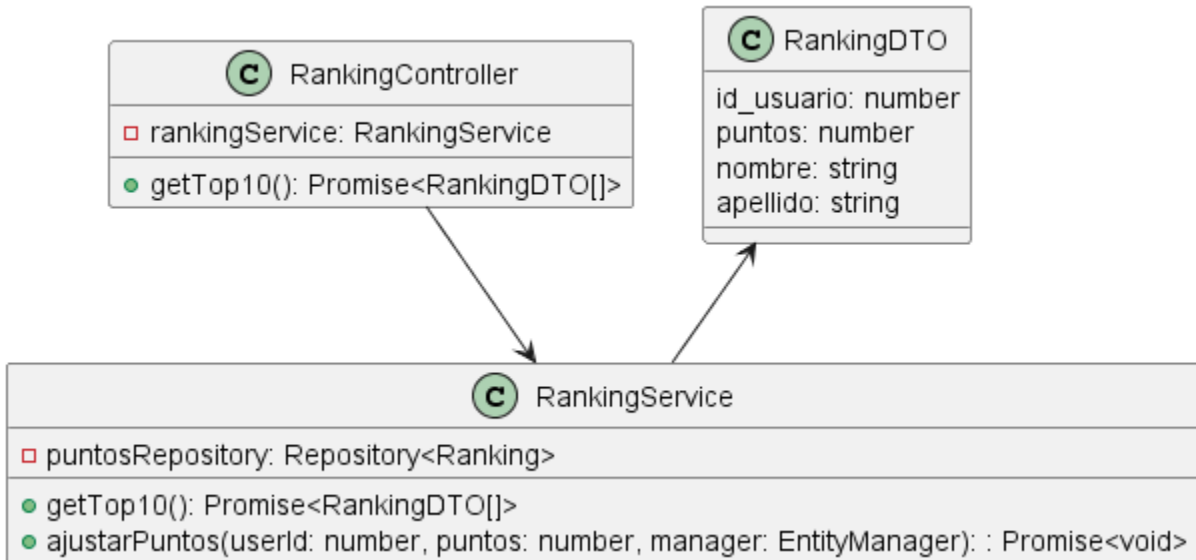
Donacion service



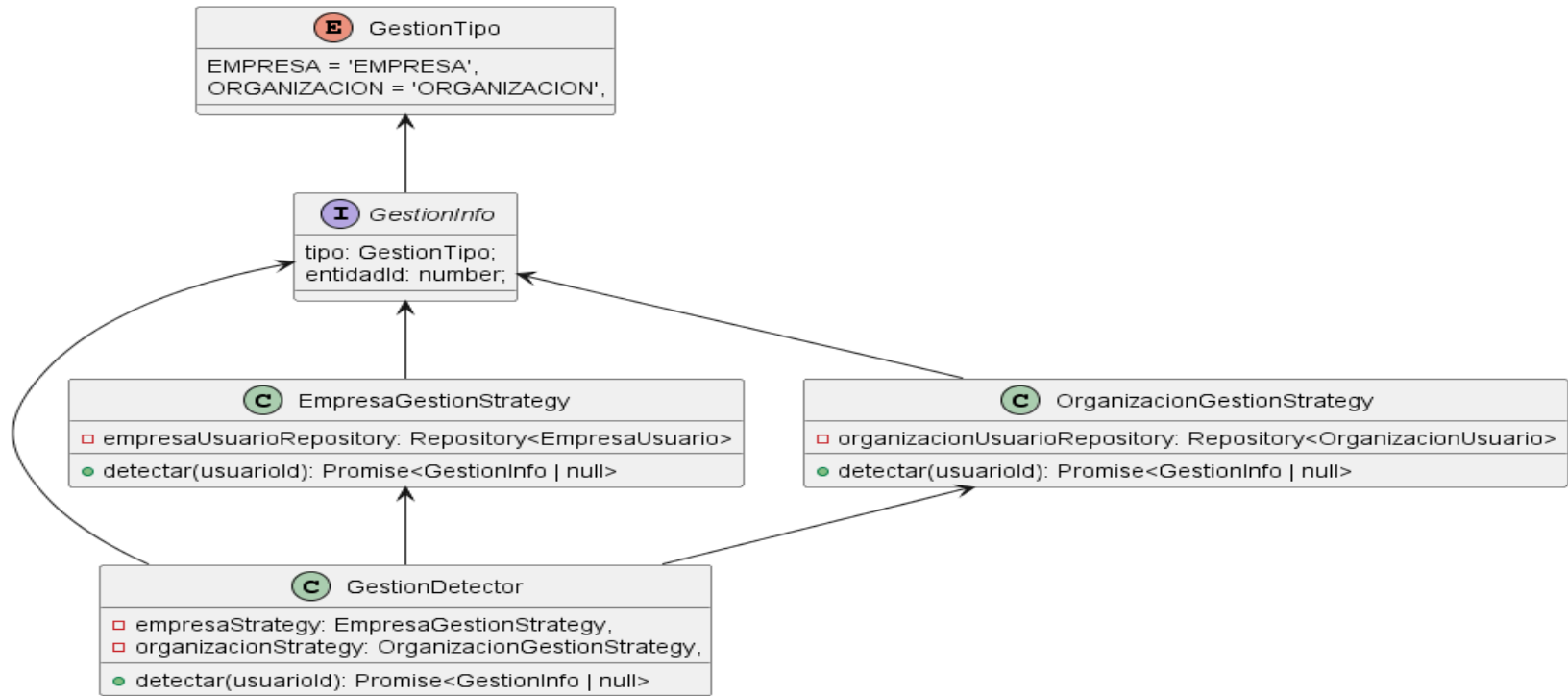
CAMPAIGN SERVICE



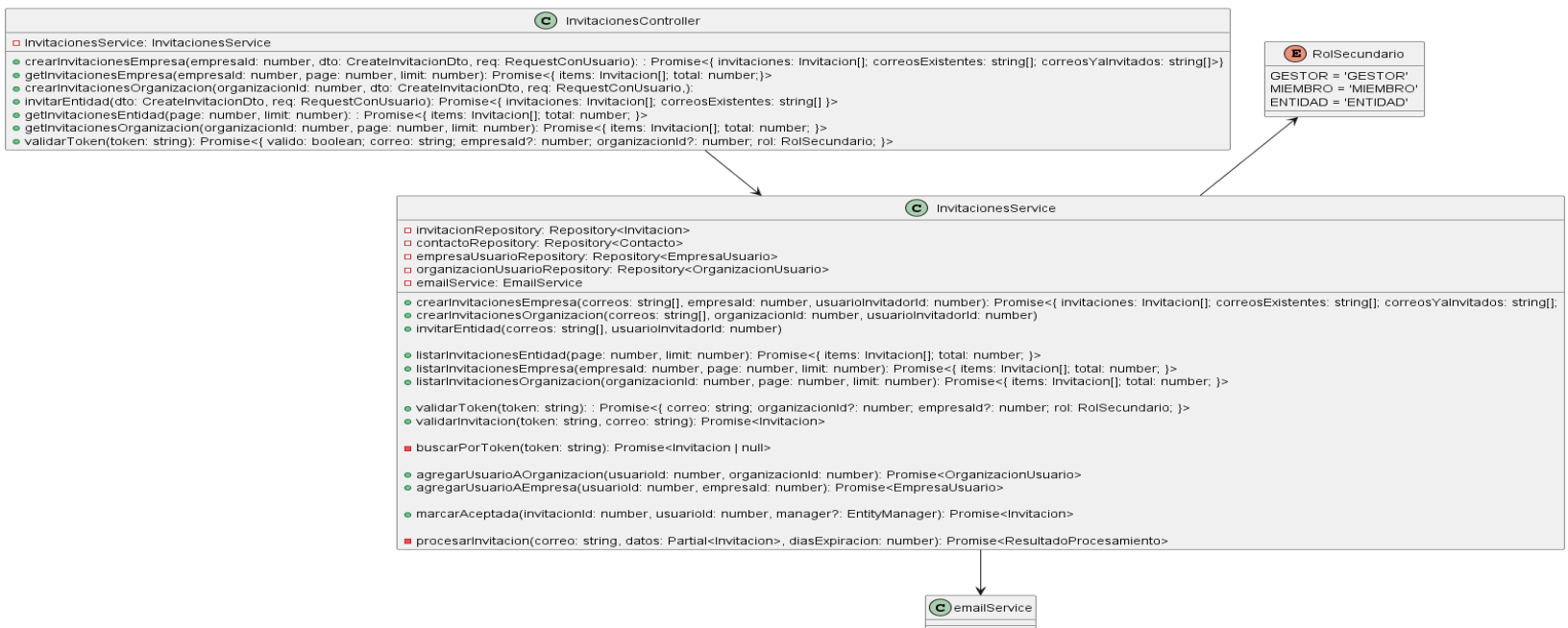
Ranking Service



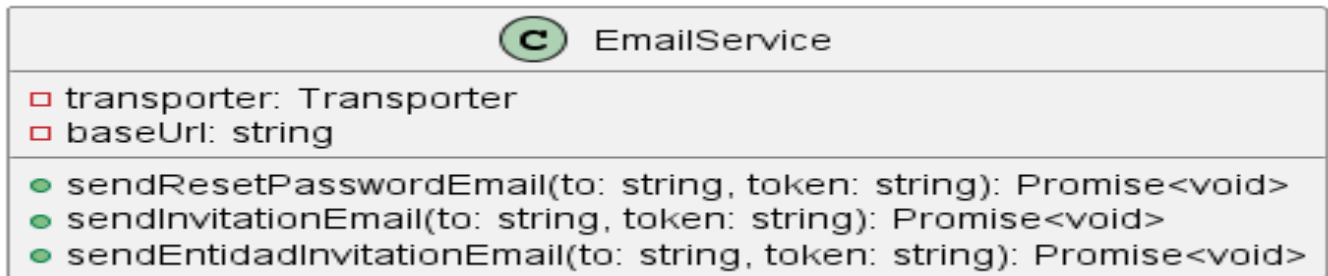
Gestion Detector



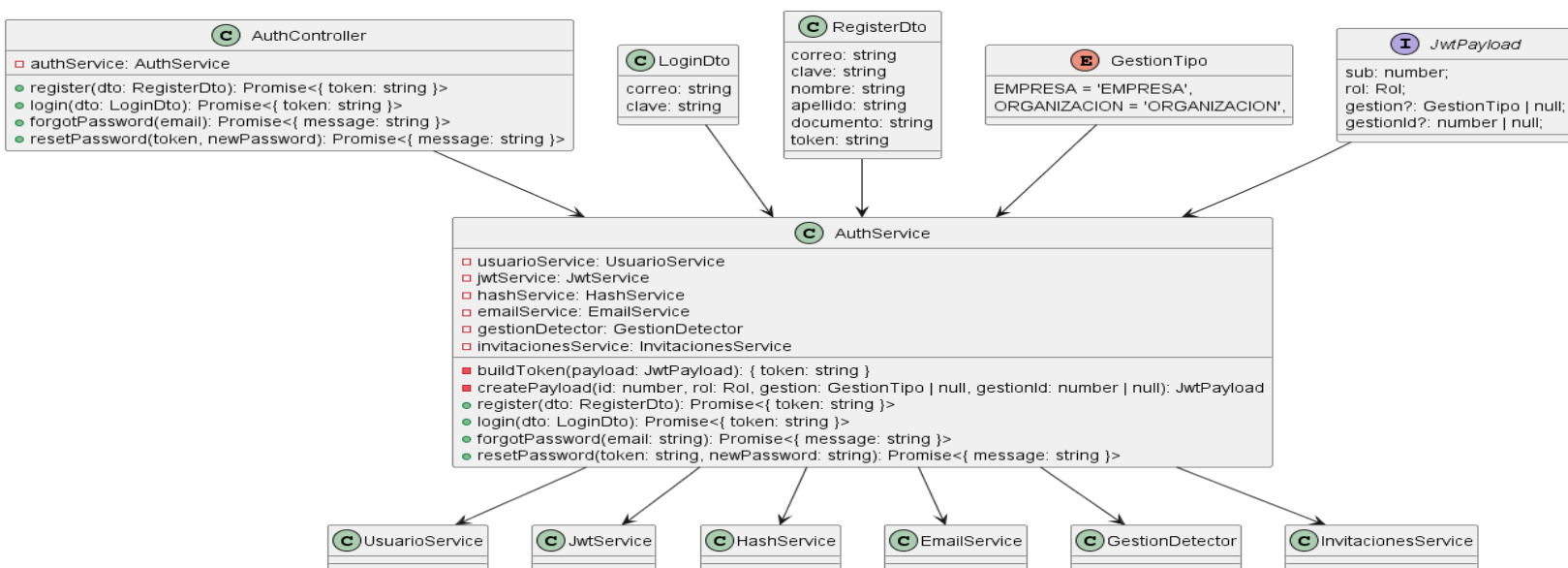
Invitaciones Service



Email Service



Auth Service



Hash Service



*Todos los diagramas se pueden visualizar en el repositorio.

Diagrama de Secuencia

En este apartado se describen dos casos de uso principales del sistema, representados mediante diagramas de secuencia. Estos permiten analizar de manera detallada la interacción entre los distintos componentes del sistema, tales como el cliente (frontend), la API desarrollada en el backend y la base de datos.

Los casos contemplados son:

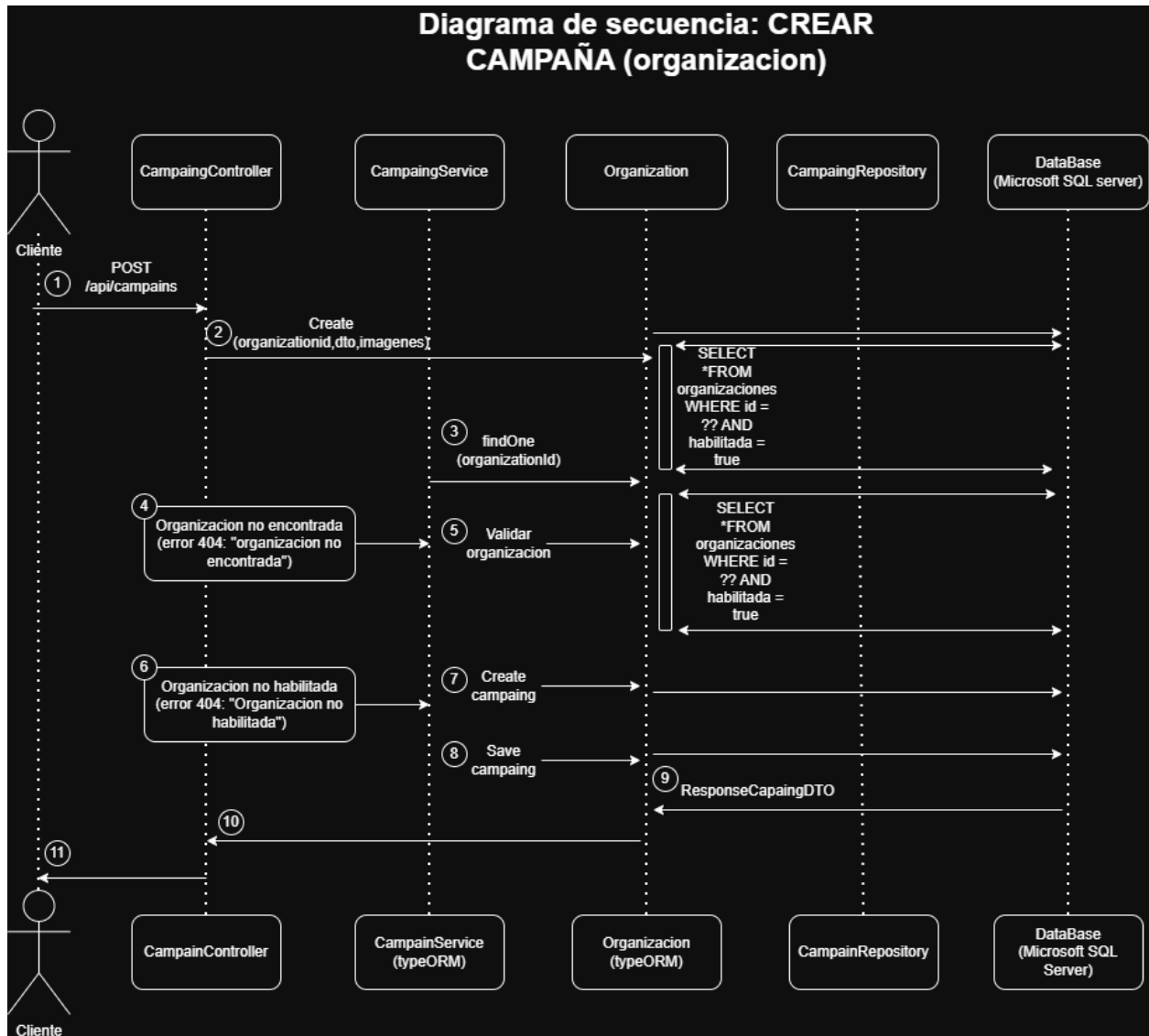
- **Creación de una campaña solidaria**
- **Realización de una donación**

Caso 1: Creación de campaña solidaria

Un usuario perteneciente a una organización registrada intenta crear una nueva campaña solidaria ingresando los datos requeridos.

El flujo del proceso es el siguiente:

1. El cliente envía una solicitud HTTP POST a la API con los datos de la campaña.
2. El controlador recibe la solicitud y delega la operación al servicio correspondiente.
3. El servicio busca la organización asociada.
4. Si la organización no existe, se retorna un error indicando que no fue encontrada y el proceso finaliza.
5. Se valida que la organización se encuentre habilitada.
6. Si la organización no está habilitada, se retorna un error y el proceso finaliza.
7. El servicio crea la entidad de campaña con los datos recibidos.
8. La campaña es persistida en la base de datos (Microsoft SQL Server).
9. El servicio construye y devuelve el ResponseCampaignDTO.
10. El controlador recibe la respuesta del servicio.
11. El controlador envía la respuesta final al cliente.



Caso 2: Realización de una donación

Un usuario autenticado en el sistema desea realizar una donación a una campaña existente.

El flujo del proceso es el siguiente:

1. El cliente inicia la creación de una donación enviando una solicitud HTTP **POST** a la API.
2. El controlador recibe la solicitud y delega la operación al servicio de donaciones.
3. El servicio busca la campaña asociada.
4. **Si la campaña no existe, se retorna un error indicando que no fue encontrada y el proceso finaliza.**
5. El servicio busca el usuario que realiza la donación.
6. **Si el usuario no existe, se retorna un error indicando que no fue encontrado y el proceso finaliza.**
7. Se valida que el monto de la donación no supere el objetivo permitido.
8. Se calcula el monto final de la donación y los puntos correspondientes.
9. El servicio crea la entidad de donación.
10. La donación persiste en la base de datos (**Microsoft SQL Server**).
11. El servicio construye y devuelve el **ResponseDonationDTO**.
12. El controlador recibe la respuesta del servicio.
13. El controlador envía la respuesta final al cliente.

Diagrama de Secuencia – Crear Donación (API)

